LLM Comparison: Bonus Project Report

Course: Software Engineering
Project: Apache Roller Design Smell Analysis
Date: 2026-02-17
Analyzed Component: `Weblog.java` and related architecture

---

1. Introduction

This report compares the performance of two Large Language Models (LLMs):

- **Claude 3.5 Sonnet**
- **Gemini Pro 1.5**

Each model was evaluated in two different operational modes:

- **Non-Agentic Mode:** Single-file analysis using direct prompt-response interaction.
- **Agentic Mode:** Autonomous exploration of the broader codebase, allowing cross-file dependency analysis and architectural reasoning.

The evaluation focuses on two primary metrics:

1. **Ability to identify design smells discovered during manual analysis.**
2. **Quality and compilability of the refactored code proposed by the LLM.**

The goal is to assess whether agentic LLMs provide more accurate and safer refactoring recommendations compared to traditional prompt-based analysis.

---

2. Metric 1: Design Smell Detection Capability

2.1 Manual Analysis Baseline

Manual inspection of the Apache Roller codebase identified the following critical design smells in the `Weblog` architecture:

1. **God Class**
   The `Weblog` class contains excessive responsibilities beyond a single domain concern.

2. **Service Locator Pattern**
   The class directly accesses services via the static `WebloggerFactory`, violating Dependency Inversion.
3. **Cyclic Dependency**
   Circular dependencies exist between domain objects and service layer components.
4. **Primitive Obsession**
   Complex concepts such as plugin configurations are represented using primitive data types (e.g., comma-separated strings).
5. **Hidden Database Queries in Getter Methods**
   Getter methods unexpectedly trigger database access, causing performance and architectural violations.
   These five smells were used as the evaluation baseline.

2.2 LLM Detection Performance Comparison

Design Smell	Gemini (Non-Agentic)	Gemini (Agentic)	Claude (Non-Agentic)	Claude (Agentic)
God Class	✅ Found	✅ Found	✅ Found	✅ Found
Service Locator	✅ Found (Local)	✅ Found (Systemic)	✅ Found (Local)	✅ Found (Systemic)
Cyclic Dependency	❌ Missed	✅ Found	❌ Missed	✅ Fully Mapped
Primitive Obsession	✅ Found	❌ Missed	✅ Found	✅ Found
Hidden DB Queries	❌ Missed	❌ Missed	❌ Missed	✅ Found
Total Score	**3/5**	**3/5**	**3/5**	**5/5**

2.3 Analysis of Results

Non-Agentic Mode Performance

Both Gemini and Claude performed similarly in non-agentic mode. They successfully identified design smells visible within the individual file, including:

- God Class
- Primitive Obsession
- Local Service Locator usage

However, both models failed to detect architectural-level problems because they lacked visibility into the broader system.

Limitations observed:

- No awareness of cross-file dependencies
- No architectural context
- No systemic pattern detection

Agentic Mode Performance

Agentic mode significantly improved detection accuracy.

Claude 3.5 Sonnet (Agentic) demonstrated superior architectural understanding by identifying:

- System-wide Service Locator usage
- Hidden database queries inside getter methods
- Complete cyclic dependency chains

These findings required analysis across multiple files and service layers.

Gemini Agentic correctly identified cyclic dependencies and systemic Service Locator usage but failed to detect hidden database access patterns.

Design Smell Detection Winner: Claude 3.5 Sonnet (Agentic)

Claude Agentic achieved a perfect detection score (5/5), demonstrating superior architectural reasoning capability.

---

3. Metric 2: Refactoring Quality and Compilability

3.1 Refactoring Challenge Context

The Apache Roller architecture relies heavily on the static `WebloggerFactory` Service Locator pattern.

This creates tight coupling across more than 30 classes. Refactoring a single class without addressing the overall architecture introduces compilation failures and runtime instability.

Therefore, safe refactoring must consider:

- Dependency structure
- Backward compatibility
- System-wide impact

3.2 Refactoring Strategy Comparison

LLM (Mode)	Proposed Strategy	Estimated Compilability	Analysis
Gemini (Non-Agentic)	Local Dependency Injection	🔴 Fails	Breaks dependencies across multiple files. Refactoring incomplete.
Claude (Non-Agentic)	Extract Services Locally	🔴 Fails	Suggests textbook refactoring without architectural awareness.
Gemini (Agentic)	Architectural Refactoring	🟡 Partial	Recognizes dependency cycle but requires complex implementation effort.
Claude (Agentic)	Phased Migration Strategy	🟢 Builds Safely	Proposes gradual migration while preserving legacy compatibility.

3.3 Refactoring Quality Analysis

Non-Agentic Refactoring

Both Gemini and Claude suggested isolated dependency injection refactoring.

However, these recommendations were unsafe because:

- They ignored system-wide Service Locator usage
- They introduced breaking changes across dependent classes

- The resulting code would not compile without additional systemic changes

These suggestions reflect theoretical correctness but practical infeasibility.

Agentic Refactoring

Claude Agentic proposed a safe phased migration strategy:

Phase 1: Maintain backward compatibility using existing Service Locator
Phase 2: Introduce dependency injection framework gradually
Phase 3: Incrementally migrate services and remove static dependencies

This approach ensures:

- Compilation safety
- Incremental migration
- Reduced system risk

This reflects real-world software engineering best practices.

Refactoring Quality Winner: Claude 3.5 Sonnet (Agentic)

Claude Agentic was the only model that proposed a realistic, safe, and implementable migration strategy.

---

4. Final Comparison Summary

Category	Gemini Non-Agentic	Gemini Agentic	Claude Non-Agentic	Claude Agentic
Design Smell Detection	Moderate	Good	Moderate	Excellent
Architectural Understanding	Poor	Good	Poor	Excellent
Refactoring Quality	Unsafe	Moderate	Unsafe	Excellent
Compilability Awareness	Poor	Moderate	Poor	Excellent

Category	Gemini Non-Agentic	Gemini Agentic	Claude Non-Agentic	Claude Agentic
Practical Applicability	Low	Moderate	Low	Very High

---

5. Final Conclusion

Overall Winner: Claude 3.5 Sonnet (Agentic Mode)

Claude Agentic demonstrated superior performance across both evaluation metrics.

Key strengths:

- Identified all manually discovered design smells
- Detected hidden architectural issues missed by other models
- Proposed safe and realistic refactoring strategies
- Demonstrated strong understanding of legacy system constraints

---

Key Insight

Non-Agentic LLM analysis is limited to surface-level code inspection and may produce unsafe refactoring recommendations.

Agentic LLM analysis enables:

- Cross-file reasoning
- Architectural pattern detection
- System-level refactoring planning

This makes agentic LLMs significantly more reliable for analyzing and refactoring complex legacy systems.